

Performance basics

COMPLETE LEARNING GUIDE

What You'll Learn

Frontend performance directly affects user experience, conversion rates, and search ranking — Google explicitly uses Core Web Vitals as a ranking signal, and studies consistently show even small increases in load time measurably increase bounce rates. This submodule covers the metrics that actually matter (Core Web Vitals: LCP, INP, CLS), the highest-leverage techniques for improving them — code splitting and lazy loading to shrink what's shipped, image optimization since images are typically the largest contributor to page weight, avoiding unnecessary re-renders in component frameworks, and understanding the critical rendering path so render-blocking resources don't delay first paint — and closes with how to actually measure performance (Lighthouse, DevTools) rather than guessing, since optimizing without measuring routinely wastes effort on things that were never actually the bottleneck.

1. Core Web Vitals: LCP, INP, CLS

Largest Contentful Paint (LCP) measures how long it takes the largest visible element (usually a hero image or heading) to render — the user's perception of 'is this page loaded yet' — and should ideally happen within 2.5 seconds. Interaction to Next Paint (INP, which replaced First Input Delay as the official responsiveness metric in 2024) measures how quickly the page responds to user interactions throughout the entire visit, not just the very first one — a good score is under 200ms. Cumulative Layout Shift (CLS) measures unexpected visual movement of content after it's already rendered (an ad loading late and pushing text down just as a user was about to tap something) — a good score is under 0.1, and it's fixed primarily by always reserving space (explicit width/height, or aspect-ratio) for images, ads, and embeds before they load.

- LCP < 2.5s, INP < 200ms, CLS < 0.1 are the current 'good' thresholds Google uses for Core Web Vitals.
- Setting width/height attributes (or CSS aspect-ratio) on every image reserves its space before the file loads, directly preventing CLS.
- Web fonts loading late and swapping in can also cause layout shift — font-display: optional or preloading critical fonts mitigates this.

2. Reducing What You Ship: Code Splitting and Tree Shaking

The single largest lever for initial load performance is usually reducing how much JavaScript the browser must download, parse, and execute before the page becomes interactive. Code splitting breaks a bundle into smaller chunks loaded only when needed (most commonly per-route, as covered in the Routing submodule, via `dynamic import()`); tree shaking is a build-time optimization where bundlers (Webpack, Vite, Rollup) statically analyze ES module imports/exports and exclude exported code that's never actually imported anywhere in the app, which only works reliably with native ES modules, not older CommonJS `require()` patterns.

JAVASCRIPT

```
// Dynamic import – this chunk is only downloaded when the user opens the modal,  
// not as part of the initial bundle  
async function openExportDialog() {  
  const { generatePdfReport } = await import('./pdfReportGenerator.js');  
  generatePdfReport();  
}
```

3. Image Optimization

Images are typically the single largest contributor to total page weight, which makes them the highest-leverage optimization target on most real pages. Practical steps: serve modern formats (WebP or AVIF, both meaningfully smaller than JPEG/PNG at equivalent visual quality, with a fallback via `<picture>` for older browsers), compress appropriately for the actual display size rather than shipping a source-resolution file, use `loading="lazy"` on below-the-fold images so the browser defers fetching them until they're near the viewport, and use `srcset/sizes` (covered in the Responsive Design submodule) so mobile devices download an appropriately smaller file rather than a desktop-sized one.

HTML

```

```

4. Minimizing Unnecessary Re-Renders

In component frameworks, re-rendering components whose output hasn't actually changed wastes CPU time and can cause visible jank, especially on lower-powered mobile devices. In React specifically, `React.memo` prevents a component from re-rendering when its props haven't changed (by reference, for objects/functions/arrays — which is why `useCallback/useMemo` on those props matters, as covered in the Hooks submodule), and correct, stable list keys (covered in the Component Architecture submodule) prevent the framework from unnecessarily discarding and recreating DOM nodes on every update. Profiling with React DevTools' Profiler tab (or the equivalent in Vue/Angular devtools) reveals exactly which components are re-rendering and why, rather than guessing.

5. The Critical Rendering Path

The critical rendering path is the sequence of steps the browser must complete before it can paint the first pixels: parse HTML, fetch and parse CSS (render-blocking by default — the browser won't paint anything until CSS is parsed, to avoid a flash of unstyled content), then build the render tree and layout. A `<script>` tag with no attributes blocks HTML parsing entirely at the point it appears, until the script downloads and executes — the `defer` attribute downloads it in parallel and executes it after parsing completes (in document order), while `async` downloads in parallel and executes as soon as it's ready (potentially out of order relative to other scripts), which is why `defer` is usually the safer default for most application scripts.

HTML

```
<!-- Blocks HTML parsing until this script downloads AND executes -->
<script src="legacy.js"></script>

<!-- Downloads in parallel with parsing, executes after parsing, in order -->
<script src="app.js" defer></script>

<!-- Downloads in parallel, executes immediately when ready (order not
guaranteed) -->
<script src="analytics.js" async></script>
```

6. Caching: Browser Cache, CDNs, and Service Workers

Repeat visits should almost never re-download unchanged assets. HTTP caching headers (Cache-Control with a long max-age plus a content hash in the filename, so the filename itself changes whenever the content does, safely invalidating the cache) let browsers reuse a previously downloaded file without even a network round-trip to check for changes. A CDN serves static assets from a server geographically close to the user, reducing round-trip latency compared to a single origin server. Service workers go further, enabling assets and even full offline functionality to be served from a local cache the app controls directly — powerful, but adding real complexity that's usually only justified for apps with genuine offline requirements.

7. Measuring Before Optimizing

Optimizing without measuring routinely wastes effort improving something that was never the actual bottleneck. Lighthouse (built into Chrome DevTools, or run via CLI/CI) audits a page against Core Web Vitals and gives specific, prioritized suggestions with estimated impact. The DevTools Performance panel records a detailed timeline of exactly what the browser was doing millisecond by millisecond during a real interaction, which is what reveals genuine bottlenecks — a slow synchronous function, an unexpected reflow, excessive re-renders — that intuition alone won't reliably find. Real User Monitoring (RUM) tools go a step further by capturing these metrics from actual visitors' real devices and networks in production, which routinely differ substantially from a fast developer machine on office WiFi.

Common Pitfalls

- Shipping one large bundle with no code splitting, forcing every user to download JavaScript for pages they may never visit.
- Not reserving space (width/height or aspect-ratio) for images and embeds, causing layout shift as they load in.
- Serving desktop-resolution images to mobile devices instead of using srcset/sizes or a modern compressed format.
- Placing a blocking `<script>` tag with no `defer/async` in the `<head>`, delaying HTML parsing and first paint.
- Optimizing based on guesswork instead of profiling with Lighthouse or the DevTools Performance panel first.
- In React, creating new inline object/array/function props on every render for a memoized child, silently defeating `React.memo`.
- Treating performance as a one-time pre-launch task instead of a metric tracked continuously via RUM in production.

Key Takeaways

- Core Web Vitals (LCP < 2.5s, INP < 200ms, CLS < 0.1) are the concrete, measurable targets for real-world perceived performance.
- Code splitting and tree shaking reduce what a user's browser has to download and execute before the page becomes usable.
- Images are usually the largest contributor to page weight — format, compression, sizing, and lazy loading are the highest-leverage fixes.
- `defer` (and sometimes `async`) on `<script>` tags prevents render-blocking JavaScript from delaying first paint.
- Always measure with Lighthouse or DevTools before optimizing — fixing an unmeasured guess wastes effort on something that may not be the real bottleneck.

Learn More

web.dev — Core Web Vitals

<https://web.dev/articles/vitals>

web.dev — Fast load times

<https://web.dev/explore/fast>

Chrome DevTools — Performance features reference

<https://developer.chrome.com/docs/devtools/performance>